

TECHNICAL DEMO REPORT

Shared memory for every AI coding agent.

How a team of AI coding tools — Claude Code, Codex, and Cursor — share one queryable knowledge base on mem9, so none starts cold or repeats another's work.

Team acme **Repos** pulumi · lza **Store** mem9 → TiDB Cloud

Access Model Context Protocol

Three tools, one brain.

A platform team runs its infrastructure as code — AWS, Cloudflare, and Okta via Pulumi, plus an AWS Landing Zone for accounts and guardrails. Increasingly that code is written by AI coding agents. The problem isn't any single agent's capability; it's that **they share no memory**. Each session starts cold, re-derives context from the filesystem, and re-makes the same mistakes.

Without shared memory



Duplication

An agent can't see a component already exists, so it builds a second one — a duplicate bucket, a parallel DNS record.



Broken conventions

It writes a raw `aws.s3.Bucket` instead of the library wrapper, skipping required tags and encryption.



Invisible blast radius

It changes a shared library and breaks resources two or three hops away that it never saw.



Cold starts

The next session — or the next tool — re-derives everything from scratch and repeats the same errors.

The fix. One shared, queryable memory in TiDB Cloud — reached through mem9 — that every agent reads **before** it builds and writes back **when** it's done. A single standing instruction makes the behavior automatic across all three tools: **query first → compose libraries → write back.**

Team = space. Repo = namespace.

mem9 maps cleanly onto how teams already organize work. A **team** is a mem9 **space** — one API key, one isolated TiDB Cloud cluster behind it. A **repo** is an `appId` namespace inside that space, so multiple repos share a cluster but keep separate recall scopes.

acme · pulumi

`appId acme_pulumi_kb`

AWS · Cloudflare · Okta IaC. 21 components: reusable libraries plus production and staging instances.

acme · lza

`appId acme_lza_kb`

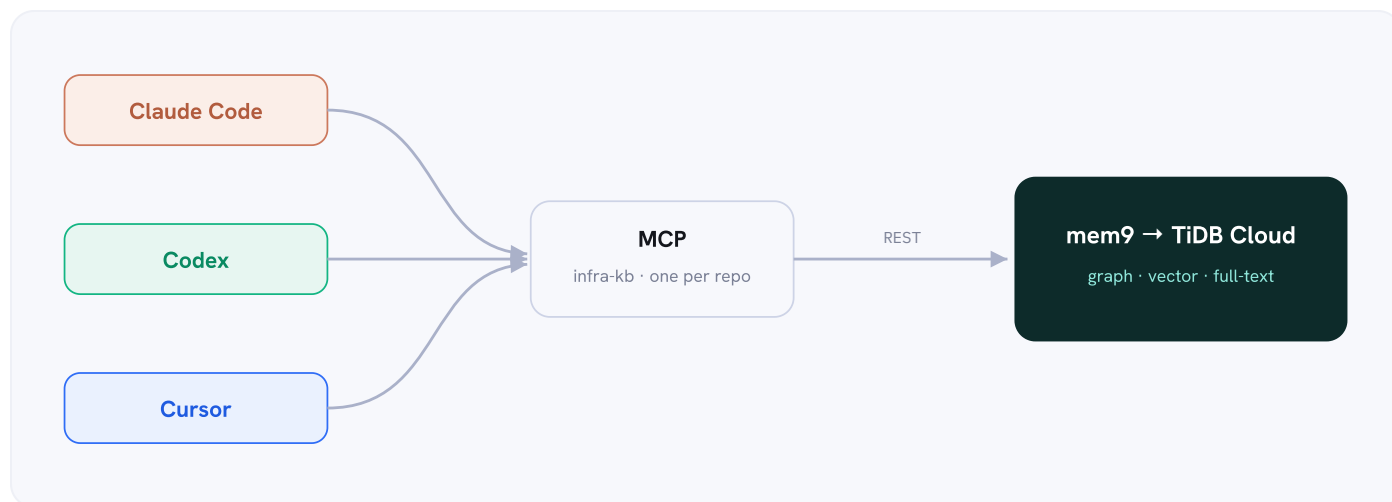
AWS Landing Zone: accounts, OUs, SCPs, IAM baseline. 11 components modelling the org layer.

Isolation & cross-repo

- **Cross-repo** within a team: an `account_ref` links LZA accounts to the Pulumi stacks that live in them — recalled from both namespaces and merged client-side.
- **Cross-team** isolation: a different team is a different space with a different API key. One team cannot read another's memories — enforced by mem9 at the space level, not by application code.

In production each team's space sits on its own managed TiDB Cloud cluster. Agents never handle raw database credentials — the mem9 API key is the only secret.

One store. One plug. One key.



MCP is the plug

Every tool loads the same Model Context Protocol config — one named server per repo. No custom glue per tool.

One store, every pattern

Relational state, the dependency graph, vector embeddings, and full-text all live in the same memory. No ETL between four systems.

Just an API key

mem9 provisions and manages the TiDB Cloud cluster. Agents read and write through one key and never touch credentials.

The standing instruction

Loaded into every tool's rules file (`CLAUDE.md` / `AGENTS.md` / `.cursorrules`), so the read-first / write-back behavior happens on every run with no per-task reminder:

Before writing infrastructure code, query the infra-kb MCP:

- what already exists (don't duplicate)
- how things connect (dependency edges)
- what previous sessions did (session log)

Never use raw `aws.*` / `cloudflare.*` / `okta.*` — compose the libs/ wrappers.

After any change, record it with `write_component` so the next session — and the next tool — starts warm.

mem9

Infra Knowledge Base

acme · pulumi + lza

TiDB Cloud

Reset

Live

Dependencies

Search

Config

Behavior is configuration, not code.

One standing instruction in every tool's rules file: **query first** → **compose** → **write back**.

STANDING INSTRUCTION · CLAUDE.MD / AGENTS.MD / .CURSORRULES

Before writing infrastructure code, query the infra-kb MCP:

- what already exists (don't duplicate)
- how things connect (dependency edges)
- what previous sessions did (session log)

Never use raw `aws.*` / `cloudflare.*` / `okta.*` — compose the libs/ wrappers.
Follow `acme-<env>-<name>` naming and the required tags.

After any change, record it with `write_component` so the next session — and the next tool — starts warm.

MCP SERVERS · ONE ENTRY PER REPO

```
{
  "mcpServers": {
    "infra-kb-pulumi": { "command": "python", "args": ["-m", "src.mcp_server"] },
    "infra-kb-lza": { "command": "python", "args": ["-m", "src.mcp_server"] }
  }
}
```

`query_knowledge_base` · `write_component` → hybrid recall + atomic writes

[DASHBOARD](#) · [CONFIG](#)

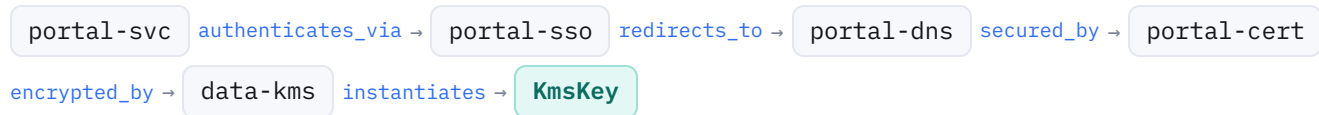
The **Config** tab: the standing instruction every tool loads, and the one-entry-per-repo MCP wiring that makes it automatic.

The question similarity can't answer.

A vector store finds things that *look* related. It cannot compute a transitive closure — "what, exactly, breaks if I change this?" That is graph reachability, and it is what recursive traversal over the dependency graph gives you. The knowledge base records each `depends_on` and every composition edge; mem9 holds them, and a single traversal walks the chain to any depth.

A five-hop chain

The production customer-portal service sits at the top of a chain that bottoms out at a single KMS key — five hops down, across four resource types:



mem9

Infra Knowledge Base

acme · pulumi + 12a

TiDB Cloud

Reset

Live

Dependencies

Search

Config

Transitive dependencies, in one pass.

Recursive reachability over the dependency graph — the question a vector store can't answer. Pick a component: trace what it **needs**, or what **breaks** if it changes. Each hop is a column.

Blast radius

Dependencies

acme-prod-portal-svc

TRY

rotate KmsKey →

portal-svc needs →

account-prod →

9 components · 4 hops · downstream

src

portal-svc

production

HOP 1 3

d1

portal-dns

production

d1

portal-sso

production

d1

Service

library

HOP 2 3

d2

portal-cert

production

d2

DnsRecord

library

d2

SsoApplication

library

HOP 3 2

d3

data-kms

production

d3

TlsCertificate

library

HOP 4 1

d4

KmsKey

library

LONGEST CHAINS

5h portal-svc → authenticates_via → portal-sso → uses → portal-dns → secured_by → portal-cert → instantiates → TlsCertificate → encrypted_by → KmsKey

5h portal-svc → authenticates_via → portal-sso → uses → portal-dns → secured_by → portal-cert → encrypted_by → data-kms → instantiates → KmsKey

4h portal-svc → fronted_by → portal-dns → secured_by → portal-cert → instantiates →

HOW IT'S RESOLVED

Dependencies of 'acme-prod-portal-svc': 9 components across 4 hop(s), transitively (acme-prod-portal-svc needs ...). Graph reachability over mem9 (appId=acme_pulumi_kb_demo).

DASHBOARD · DEPENDENCIES

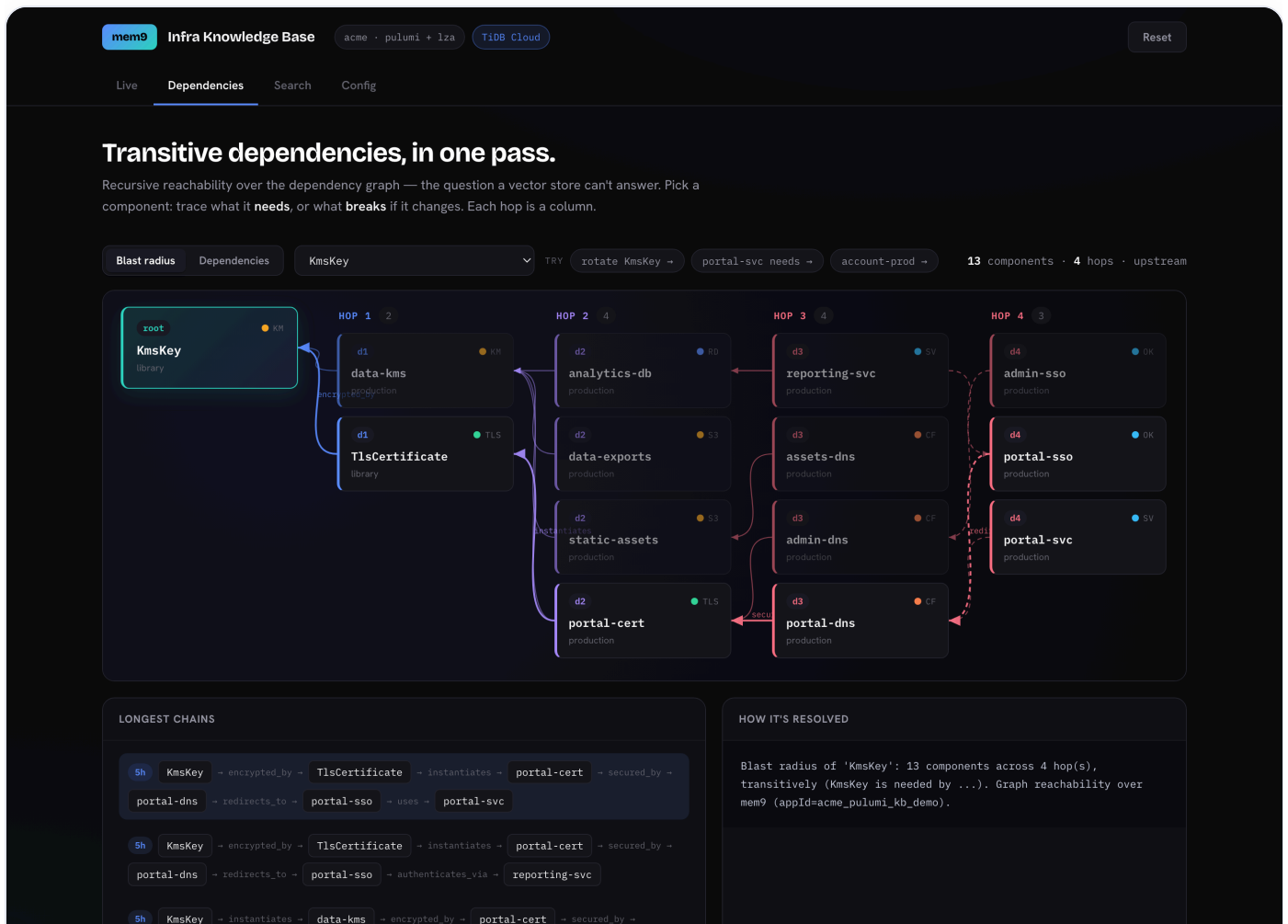
"What does **portal-svc** need?" — each hop is a column; the chain reads left-to-right and converges on the shared KMS foundation. 9 components, 4 hops downstream.

Blast radius of one KMS key

Run the traversal in reverse from **KmsKey** — "what breaks if we rotate it?" — and the answer is **13 components across four depths**. No flat search or similarity ranking surfaces this; only walking the graph does.

DEPTH	COUNT	COMPONENTS REACHED
d1	2	data-kms, TlsCertificate
d2	4	portal-cert, analytics-db, data-exports, static-assets
d3	4	portal-dns, admin-dns, assets-dns, reporting-svc
d4	3	portal-sso, portal-svc, admin-sso

Why it matters. Before touching a shared resource, an agent (or a human) sees the full transitive closure of what depends on it — and stages the change accordingly. The cost of a blind change is exactly the surprise this prevents.



DASHBOARD · DEPENDENCIES

"What breaks if we rotate **KmsKey**?" — the reverse traversal fans 13 components across four depths, all funneling onto the anchored foundation. The Trace panel reads each chain as a sentence.

The live dashboard renders this as a depth-laned "Dependency Rail": each hop is a column, connectors carry the relationship, and the foundation node is anchored as everything funnels toward it.

Hybrid recall, and why one engine.

Vector + full-text, one store

The same memory that holds the graph also answers semantic and keyword queries. Ask *"what encrypts production data"* and mem9 returns the KMS key, the resources it encrypts, and the TLS cert — ranked by hybrid relevance, served from TiDB Cloud with no separate vector database and no synchronization to keep in step.

mem9

Infra Knowledge Base

acme · pulumi + 12a

TiDB Cloud

Reset

Live

Dependencies

Search

Config

Vector + full-text, one store.

Semantic and keyword recall over the same memory — served by mem9, no separate vector DB.

what encrypts production data

Hybrid

Vector

Full-text

Search

admin portal access control

where do database backups live

what encrypts production data

RESULTS

6 results · hybrid

#1

acme-prod-admin-ss

Okta

production

pulumi

score 1.629

The acme-prod-admin-ss component uses Okta for Single Sign-On (SSO) in the production environment.

#2

acme-prod-admin-dns

Cloudflare

production

pulumi

score 1.894

The acme-prod-admin-dns resource uses Cloudflare for DNS in production.

#3

acme-prod-data-exports

S3

production

pulumi

score 4.384

The acme-prod-data-exports resource is an S3 bucket used for production data exports.

#4

acme-prod-analytics-db

RDS

production

pulumi

score 1.676

The acme-prod-analytics-db instance hosts a production analytics Postgres database with multi-AZ configuration.

#5

PostgresDatabase

Library

library

pulumi

score 1.778

The PostgresDatabase component uses Multi-AZ configuration in production and single-AZ in staging.

#6

acme-prod-assets-dns

Cloudflare

production

pulumi

score 2.027

The acme-prod-assets-dns resource uses Cloudflare in production.

HOW IT'S RESOLVED

Hybrid = vector + full-text (served by mem9.ai / TiDB Cloud).

DASHBOARD · SEARCH

The **Search** tab: hybrid (vector + full-text) recall over the same store, with per-result type, environment, and repo, scored by relevance.

Why mem9 on TiDB — versus the alternatives

APPROACH	GRAPH REACHABILITY	SEMANTIC RECALL	ATOMIC AGENT STATE
S3 / filesystem	no traversal	no	no transactions
Vector DB only	similarity $\neq$ reachability	yes	no
Four separate systems	partial, with ETL	yes	no cross-system atomicity
mem9 $\rightarrow$ TiDB Cloud	recursive traversal	vector + full-text	one transaction

Component, edge, and session-log entry are written together. Relational state, the dependency graph, vector embeddings, and full-text indexes are one engine — so the agent's memory is consistent across concurrent writes from all three tools.

S3 is the filing cabinet for artifacts. mem9 is the brain agents query.

Closing the staging gap, warm.

Production is complete; staging has drifted — it's missing its DNS and SSO layer (four components). Three different tools bring staging to parity, each reading the shared memory before it builds and writing back when it's done. No agent starts cold.

1 • Claude Code

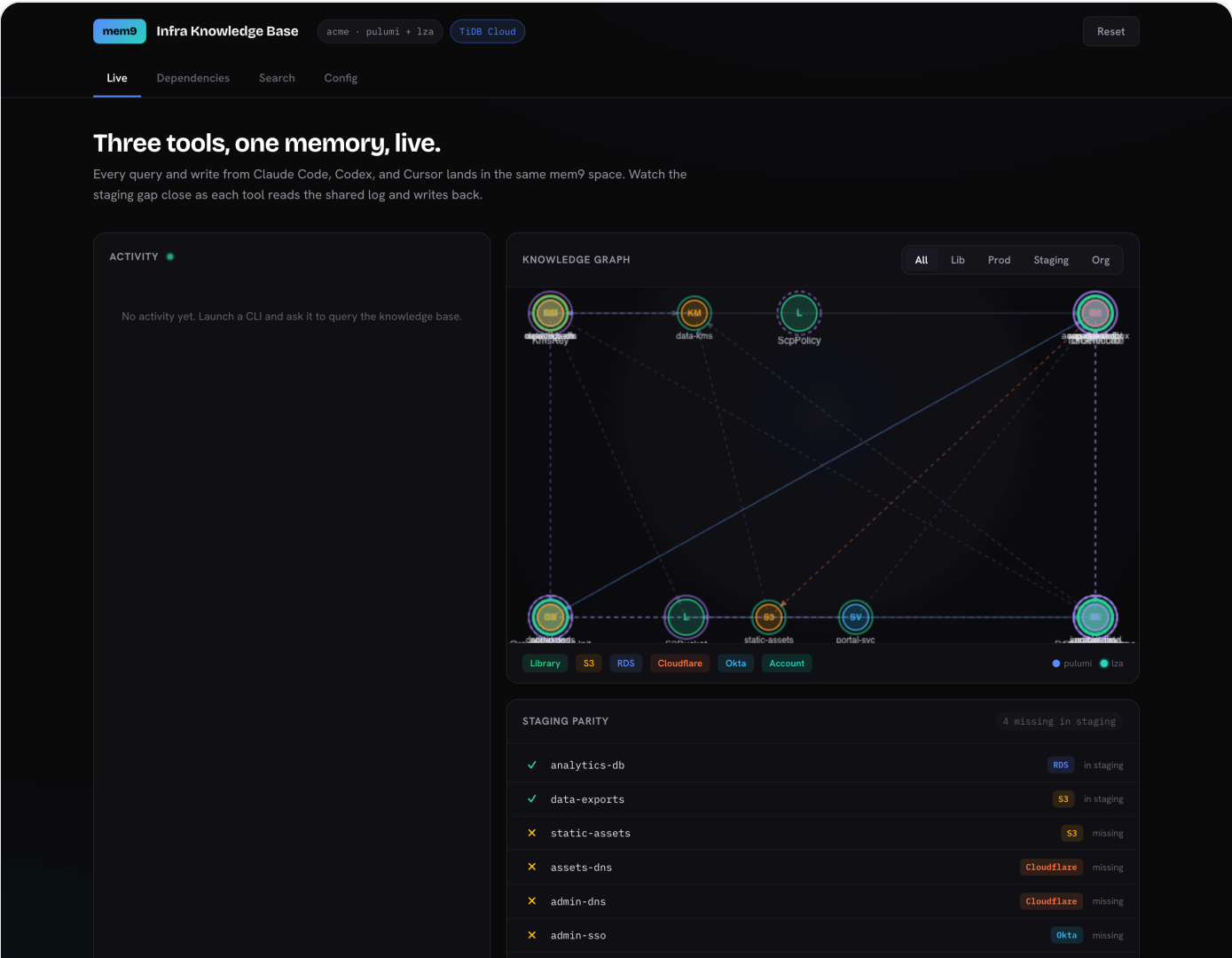
Queries the KB, finds staging missing static-assets + DNS, composes the libraries, writes both back. **Gap 4 → 2.**

2 • Codex

Opens cold with zero context. Reads the shared session log, sees what was just built, continues with admin-DNS. **Gap 2 → 1.**

3 • Cursor

Traces what prod's admin-SSO depends on, confirms it must redirect to a DnsRecord, builds the matching SSO app. **Gap 1 → 0.**



DASHBOARD · LIVE The **Live** tab: the activity feed (every tool's reads and writes), the shared knowledge graph, and the staging-parity strip — four components missing, before the agents run.

Each tool attributes its work in the activity feed; the parity strip turns green as the gap closes. The same component graph, dependency traversal, and hybrid search shown in this report are all live views over the one mem9 space — the dashboard is an observability window on the system, not the system itself.